AUDIT TECHNIQUE ET FONCTIONNEL

Gastronomie Créative (GC) · Projet fil rouge CDA · Jalon 1

Date : 2 novembre 2026 Base examinée : GC_MVC.zip et cahier des charges « Projet Fil rouge | Partage de recettes | Jalon 1 » (18 pages, daté du 24/08/2026).

Finalité : centraliser les constats des cinq étapes d'audit, détailler les écarts au cahier des charges et proposer une liste de corrections et de vérifications avant remise. Ce document est un plan de travail, pas une attestation de conformité.

1. Périmètre, méthode et limites

• Analyse statique du code C#, Razor, JavaScript, CSS, configuration et arborescence de l'archive ; comparaison avec les exigences des pages 11 à 18 du cahier des charges.

• Aucun test de navigation, de base de données active, de publication externe, de sécurité offensive ou de responsive réel n'a été exécuté pour ce document.

• Statuts : « présent » = mécanisme repéré ; « partiel » = fonction existante mais incomplète ; « absent de l'archive » = non démontré ; « à tester » = validation en exécution indispensable.

• Priorités : P0 = indispensable pour un parcours exigé ou une faille bloquante ; P1 = sécurité/fiabilité/conformité importante ; P2 = validation technique et finition ; Bonus = hors obligation.

2. Synthèse exécutive

Le projet dispose d'une ossature ASP.NET Core MVC (.NET 10), d'un contexte EF Core relié à PostgreSQL, de modèles et ViewModels distincts, d'une authentification par cookies, d'un hachage des mots de passe et d'une recherche dynamique Fetch. Les parcours de consultation de recettes et une partie de la création/modification sont implémentés.

La conformité complète du jalon 1 n'est toutefois pas démontrée : administration, publication des avis, recherche par ingrédients et filtres avancés, étapes de préparation à la création, photographie, difficulté sur 10, traitement des erreurs, déploiement, CI/CD et tests restent à réaliser ou à prouver. Les points ci-dessous indiquent les preuves attendues.

Domaine	État d'audit	Risque principal
Architecture / PostgreSQL	Présent, vérifications BDD nécessaires	Relations et contraintes non prouvées en base
Connexion / inscription	Présent, tests à compléter	Droits administrateur absents
Recettes / avis	Partiel	Parcours de création, édition, notation incomplets
Recherche Fetch	Partiel	Critères requis non appliqués
Sécurité	Protections présentes, lacunes	Jetons, configuration, validation, HTTPS

Domaine	État d’audit	Risque principal
DevOps / déploiement	Non démontré dans le ZIP	Absence de fichiers de déploiement et de pipeline

3. Matrice de conformité au cahier des charges

Exigence	État	Preuve / fichier
Inscription email, identifiant, mot de passe	Présent ; tester	AccessController.cs, UserService.cs
Connexion identifiant OU email	À vérifier en exécution	AccessController.cs, UserService.cs
Recettes par catégories	Présent ; tester	CategoriesController.cs, vues
Accueil et recettes de l'utilisateur	Présent ; tester	HomeController.cs, MesRecettes
Recherche par nom	Présent	RechercheController.cs
Recherche par ingrédient	À compléter	RechercheController.cs
Filtres catégorie, durée, difficulté	Partiel	RechercheController.cs, Recherche/Index.cshtml
Fetch API	Présent	wwwroot/JS/recherche.js
Vignettes avec photo, nom, note moyenne	Partiel	Recherche/Index.cshtml, recherche.js
Détail : ingrédients, quantités, temps	Présent ; tester	RecettesController.Detail, Detail.cshtml
Étapes repliables/dépliables	Non démontré	Detail.cshtml, scriptDetailRecette.js
Noter/commenter une autre recette	À implémenter	AvisController.cs, Detail.cshtml
Création/édition : étapes	À implémenter	RecetteCreationViewModel.cs, contrôleur
Difficulté de 1 à 10	Non conforme (1 à 5)	RecetteCreationViewModel.cs
Photo facultative	Non implémentée de bout en bout	CreerRecette.cshtml, contrôleur
Édition par créateur	Présent ; à tester	RecettesController.cs
Administrateur : toutes les recettes et utilisateurs	Absent de l'archive	UtilisateursController.cs, claims
Pas de suppression au jalon 1	À respecter	Ne pas ajouter comme exigence
Cookies : information utilisateur	À vérifier	Layout / pages légales
Données utilisateurs non divulguées	À tester	Vues, JSON, routes
Responsive desktop/tablette/mobile	À tester	styleResponsive.css
Au moins six pages MVC	Structure présente ; parcours à tester	Views / Controllers
Meta title, keywords, description	À vérifier	_Layout.cshtml, vues

Exigence	État	Preuve / fichier
404, URL directe et rewrite	À vérifier / compléter	Program.cs, routes
Jeu d'essai PostgreSQL / DBeaver	À démontrer	BDD externe au ZIP
Docker / Compose / hébergement externe	Non démontré	Pas de Dockerfile/compose fourni
GitLab CI/CD	Non démontré	Pas de .gitlab-ci.yml fourni

4. Constats détaillés et actions correctives

F01 · Modification : identifiant dans le formulaire — P0 · confirmé dans le ZIP

Fichiers : Views/Recettes/ModifierRecette.cshtml ; RecettesController.cs
Constat : Le formulaire POST n'a pas asp-route-id alors que l'action attend int id ; un champ IdRecette est présent, mais son nom ne correspond pas au paramètre id.
Action : Ajouter asp-route-id="@Model.IdRecette" (ou revoir explicitement le binding), conserver les contrôles d'autorisation serveur.
Validation : Modifier une recette et vérifier que la bonne ligne est mise à jour ; tester un ID falsifié.

F02 · Difficulté de 1 à 10 — P0 · écart confirmé

Fichiers : ViewModels/RecetteCreationViewModel.cs ; vues création/modification ; Detail.cshtml
Constat : La validation [Range(1,5)] et l'interface sur cinq toques divergent de la plage 1 à 10 exigée pour la difficulté. La note utilisateur, elle, reste de 1 à 5 étoiles.
Action : Aligner ViewModel, contrôleur, affichage et données existantes sur 1–10, sans modifier l'échelle des avis.
Validation : Créer puis modifier une recette de difficulté 10 ; refuser 0 et 11.

F03 · Étapes de préparation : création, édition et consultation — P0 · partiel

Fichiers : RecetteCreationViewModel.cs ; RecettesController.cs ; Detail.cshtml
Constat : Le détail lit la table Etapes, mais le ViewModel de création ne porte pas de collection d'étapes et le parcours de saisie n'est pas complet.
Action : Ajouter un modèle de saisie ordonné, insertion/mise à jour transactionnelle, validation du contenu et affichage repliable.
Validation : Créer trois étapes, modifier leur ordre et leur texte, recharger la recette, tester ouvrir/fermer les détails.

F04 · Ingrédients, quantités et catégories — P0 · partiel

Fichiers : RecettesController.cs ; RecetteCreationViewModel.cs ; IngredientRecetteViewModel.cs ; scriptCreerRecette.js
Constat : La sélection dynamique existe, mais la validation des quantités et des listes est incomplète ; une recette de test peut être créée sans ingrédients. Des identifiants dupliqués risquent de violer la clé composite.
Action : Exiger les champs métier pertinents, dédoublonner ou rejeter les IDs, vérifier l'existence des références, restaurer les listes disponibles après ModelState invalide.
Validation : Tester zéro ingrédient, doublons, quantité vide, ID inexistant et correction d'un formulaire invalide.

F05 · Photo facultative de recette — P0 · non abouti

Fichiers : Views/Recettes/CreerRecette.cshtml ; RecettesController.cs
Constat : L'interface évoque l'ajout d'image, sans chaîne de téléversement complète démontrée.
Action : Prévoir input file, multipart/form-data, validation taille/type, nom serveur aléatoire, stockage sûr, affichage et image par défaut.
Validation : Téléverser JPG/PNG valide ; refuser format non autorisé et taille excessive.

F06 · Publication de notes et commentaires — P0 · absent de l'archive

Fichiers : Controllers/AvisController.cs ; Views/Recettes/Detail.cshtml
Constat : Le détail affiche les avis existants, mais la page ne comporte pas de formulaire opérationnel et AvisController ne fournit pas d'action d'enregistrement.
Action : Ajouter POST authentifié et anti-CSRF ; note 1–5 ; vérifier que l'auteur n'évalue pas sa propre recette ; traiter un avis déjà existant selon la règle métier.
Validation : Publier un avis, recalculer la moyenne, tester auteur de la recette, anonyme et doublon.

F07 · Administration et rôles — P0 · absent de l'archive

Fichiers : Controllers/AccessController.cs ; UtilisateursController.cs ; RecettesController.cs
Constat : Aucun rôle administrateur exploitable dans les claims, aucune page de liste des utilisateurs ni autorisation d'éditer toutes les recettes.
Action : Définir la source fiable du rôle en BDD, le claim/les policies, une interface réservée et les contrôles serveur [Authorize] adaptés.
Validation : Tester anonyme, utilisateur normal et administrateur sur toutes les URLs et POST.

F08 · Recherche : ingrédients, filtres et résultats — P0 · partiel

Fichiers : Controllers/RechercheController.cs ; Views/Recherche/Index.cshtml ; wwwroot/JS/recherche.js
Constat : Index/Resultats acceptent seulement terme et categorie ; la recherche sur le nom et le filtre catégorie fonctionnent dans le code, mais pas les critères ingrédients, durée et difficulté. Le JSON dynamique ne présente pas la note moyenne.
Action : Étendre les paramètres et la requête EF, calculer la moyenne, transmettre les filtres dans Fetch et afficher des vignettes homogènes.
Validation : Comparer résultats GET et Fetch pour mot-clé ingrédient, catégorie, durée, difficulté et combinaisons.

F09 · Fiabilité du JavaScript de recherche — P1 · confirmé

Fichiers : wwwroot/JS/recherche.js ; Views/Recherche/Index.cshtml
Constat : Gestion d'erreur réseau/HTTP insuffisante ; incohérence name="niveaudiff" pour la cinquième difficulté ; insertion DOM perfectible.
Action : Gérer response.ok, catch et état vide ; corriger le nom du champ ; centraliser la création des cartes.
Validation : Tester serveur indisponible, aucune correspondance, filtres et navigation vers une fiche.

F10 · Sécurité des secrets et des jetons — P1 · confirmé

Fichiers : Services/EmailService.cs ; Services/TokenService.cs
Constat : Un mot de passe SMTP de repli figure dans le code ; le vecteur d'initialisation AES est réutilisé. Ces valeurs ne doivent pas être diffusées ni considérées comme sûres en production.
Action : Supprimer les secrets de repli, utiliser variables d'environnement / User Secrets ; étudier ASP.NET Core Data Protection ou un schéma authentifié avec nonce unique.
Validation : Démarrage sans secret = échec explicite ; test de génération/validation de jeton et de refus d'un jeton altéré.

F11 · Cookies, HTTPS et autorisations — P1 · à compléter

Fichiers : Program.cs ; AccessController.cs ; vues
Constat : Cookies d'authentification présents, mais paramètres HTTPS/production et information des visiteurs à confirmer ; absence de UseHttpsRedirection dans Program.cs.
Action : Vérifier proxy/terminaison TLS, Secure/HttpOnly/SameSite, expiration, consentement/information sur cookies nécessaires ; auditer toutes les actions sensibles.
Validation : Inspecter Set-Cookie en HTTPS ; tenter des actions protégées sans session.

F12 · Erreurs, 404 et routes — P1 · confirmé / à tester

Fichiers : Program.cs ; Controllers/HomeController.cs
Constat : UseExceptionHandler("/Home/Error") pointe vers une action non repérée ; gestion personnalisée de 404 et réécriture d'URL non démontrées.
Action : Créer une action/vue d'erreur sans fuite technique ; définir traitement 404 et routes lisibles selon les attentes du formateur.
Validation : Ouvrir une URL inexistante et provoquer une erreur contrôlée en environnement de test.

F13 · HTML, accessibilité et affichage — P2 · confirmé / à tester

Fichiers : Views/Shared/_Layout.cshtml ; vues recettes ; CSS
Constat : script.js est inclus deux fois ; certaines vues imbriquent <main> dans le <main> du layout ; responsive seulement partiellement prouvable statiquement.
Action : Supprimer le doublon, conserver un seul main, vérifier labels, alternatives textuelles, menus clavier et styles sur trois formats.
Validation : Tests navigateur 360, 768 et 1440 px ; validation HTML et parcours clavier.

F14 · Relations EF et cohérence PostgreSQL — P1 · à vérifier

Fichiers : Data/GCContext.cs ; modèles ; base PostgreSQL
Constat : Les clés composites sont configurées ; les contraintes de clés étrangères réelles et les règles de suppression ne sont pas démontrées par l'archive.
Action : Inspecter schéma dans DBeaver ; documenter PK/FK, nullabilité, contraintes et transactions ; corriger uniquement les divergences observées.
Validation : Exécuter requêtes de contrôle, tests d'intégrité et sauvegarde/restauration.

F15 · Métadonnées, navigation et données privées — P2 · à vérifier

Fichiers : _Layout.cshtml ; vues ; contrôleurs
Constat : La présence de pages ne prouve ni la qualité des métadonnées title/description/keywords, ni le bon fonctionnement des liens directs, ni la non-divulgation des données utilisateurs.
Action : Vérifier chaque page et les réponses JSON ; limiter les données exposées ; vérifier le menu responsive.
Validation : Contrôler source HTML, URLs directes, déconnexion et réponse réseau.

F16 · Déploiement, CI/CD et tests — P1 · non démontré dans l’archive

Fichiers : Racine du projet ; GC_MVC.csproj
Constat : Pas de Dockerfile, Compose, projet de tests ou .gitlab-ci.yml identifiés dans l’archive ; publication externe non vérifiée.
Action : Créer la procédure reproductible de déploiement, conteneuriser si requis, configurer CI GitLab et une campagne de tests documentée.
Validation : Pipeline réussi, lancement propre sur autre machine, accès HTTPS externe et preuves de tests.

5. Plan de tests d’acceptation avant remise

ID	Scénario	Critère de réussite
T01	Inscription : valides, doublons, champs invalides	Compte créé seulement si données conformes
T02	Connexion : identifiant ou email, mauvais mot de passe, déconnexion	Session correcte et refus des erreurs
T03	Consultation : accueil, catégories, détail, auteur	Informations exactes et liens opérationnels
T04	Recherche : nom, ingrédient, catégorie, durée, difficulté	Résultats cohérents en GET et Fetch
T05	Création : champs, ingrédients, catégories, étapes, photo	Recette complète persistée en base
T06	Modification : propriétaire, autre utilisateur, admin	Droits appliqués côté serveur
T07	Avis : 1–5, commentaire, moyenne, auteur interdit	Publication et calcul exacts
T08	Administration : liste utilisateurs, modification globale	Accès réservé au rôle admin
T09	Sécurité : CSRF, validation, URL directe, cookies	Requêtes non autorisées rejetées
T10	Navigation : 404, erreurs, métadonnées, menus	Comportement correct sur toutes les pages
T11	Responsive : mobile, tablette, desktop	Aucun débordement bloquant ; parcours utilisables
T12	BDD : jeu d’essai, intégrité, sauvegarde/restauration	Données cohérentes dans DBeaver
T13	Déploiement : Docker/Compose, CI/CD, hébergement	Reproduction et accès externe démontrés

Pour chaque test, consigner : date, version/commit, préconditions, étapes, résultat attendu, résultat obtenu, statut, capture éventuelle et correction associée. Les tests ci-dessus sont à exécuter ; ils ne sont pas déclarés réussis.

6. Ordre de traitement recommandé

Phase A - Parcours indispensables (P0). Corriger le POST de modification et l’échelle de difficulté ; terminer les étapes, ingrédients et photos ; implémenter avis et administration ; compléter la recherche. Prioriser les parcours démontrables de bout en bout.

Phase B - Fiabilité et sécurité (P1). Validation des données, autorisations, erreurs, jetons et secrets, cookies et HTTPS ; vérifier l’intégrité de la base et tester les cas négatifs.

Phase C - Preuves de conformité (P1/P2). Tests responsive, navigation, métadonnées, 404, Docker/Compose, hébergement externe et GitLab CI/CD ; capturer des preuves pour le dossier.

Phase D - Finitions. Corriger les problèmes HTML/JS/CSS et améliorer l'ergonomie sans retarder les exigences obligatoires. Les favoris, impression, pagination et carrousel restent des bonus.

7. Livrables et échéances du cahier des charges

Échéance	Livrable	Points à préparer
06/11/2026	Résumé de projet, 120 à 200 mots	Contexte, objectif, technologies, état réel
12/11/2026	Dossier technique (25 à 30 pages indicatives, hors annexes)	Besoins, gestion, architecture, fonctionnalité, 2 attaques, DevOps, conteneurs, bilan
16 au 20/11/2026	Présentation orale et démonstration	Support, parcours utilisateur, preuves, limites assumées

Sections du dossier : compétences REAC ; expression des besoins ; contexte ; organisation et suivi ; analyse et spécifications fonctionnelles ; spécifications techniques ; fonctionnalité détaillée (architecture, enchaînement des écrans, extraits de code) ; deux types d'attaques web et protections ; méthodologie DevOps ; déploiement en conteneurs ; bilan, difficultés et perspectives.

Le cahier des charges prévoit également un jeu d'essai cohérent, la validation par le formateur, la remise selon les consignes pédagogiques et un hébergement externe pour les livrables web. La suppression de recettes ou d'utilisateurs n'est pas exigée à ce jalon.

8. Liste de contrôle finale avant livraison

• ☐ Tous les parcours obligatoires ont été testés et leurs résultats consignés.

• ☐ Un compte utilisateur et un compte administrateur de démonstration sont prêts, sans divulgation de mots de passe dans le dossier.

• ☐ La création, l'édition et la consultation d'une recette complète fonctionnent avec étapes, quantités et catégories.

• ☐ La recherche par ingrédients et filtres renvoie des résultats corrects et passe par Fetch.

• ☐ Les avis ne peuvent pas être publiés par le créateur de la recette ; moyenne vérifiée.

• ☐ Les permissions sont vérifiées sur les actions serveur, pas seulement dans les vues.

• ☐ Le jeu d'essai PostgreSQL est cohérent et vérifié dans DBeaver.

• ☐ Le site est testé sur mobile, tablette et desktop ; 404, URLs directes et métadonnées contrôlées.

• ☐ Les secrets sont hors dépôt ; les cookies et HTTPS sont configurés pour le déploiement.

• ☐ Les étapes de déploiement et le pipeline CI/CD sont documentés avec preuves.

• ☐ Le résumé, le dossier et le support oral respectent les dates et consignes.

9. Références et traçabilité

Source des exigences : « Projet Fil rouge | Partage de recettes | Jalon 1 », 24/08/2026, pages 11–15 (fonctionnalités et architecture), pages 17–18 (livrables et échéances), pages 8–10 (pratiques de développement et DevOps).

Source du code : archive GC_MVC.zip transmise le 02/11/2026 ; principaux fichiers examinés : Program.cs ; Data/GCContext.cs ; Controllers/{Access,Recettes,Recherche,Avis,Utilisateurs,Home}Controller.cs ; Services/{User,Email,Token}Service.cs ; ViewModels/RecetteCreationViewModel.cs ; Views/Recettes/{CreerRecette,ModifierRecette,Detail}.cshtml ; Views/Recherche/Index.cshtml ; Views/Shared/_Layout.cshtml ; wwwroot/JS/{recherche,scriptCreerRecette}.js ; wwwroot/CSS/styleResponsive.css.

Réserve : ce rapport synthétise et précise les constats de notre audit statique. Les constats « absent de l’archive » peuvent correspondre à une configuration conservée hors ZIP. Les résultats en exécution, la base réelle, le dépôt distant et l’environnement de production doivent être vérifiés séparément avant de conclure à une conformité finale.